

Week 2 • Day 5

XGBoost Classifier & Model Comparison

Trained XGBoost and generated the three-model ROC comparison, selecting the production model

DATE	INTERN NAME	PROJECT	MENTOR
Day 12 of 28 Week 2		Customer Churn Prediction & LTV Engine	

■ Technologies Used Today

Python

XGBoost

Scikit-Learn

Pandas

Matplotlib

joblib

■ Today's Key Metrics

3

Models Compared

>0.85

XGBoost AUC-ROC

300

Boosting Rounds

2.77

Scale Pos Weight

■ Day Objectives

- Understand gradient boosting vs Random Forest
- Train XGBoost with scale_pos_weight for class imbalance
- Compare all three models: LR vs RF vs XGBoost on ROC curve
- Select and save the best model as production_model.pkl
- Document model selection rationale

■ Tasks Completed Today

1. Created src/models/xgboost_model.py
2. Calculated scale_pos_weight = $5174/1869 = 2.77$ (negative/positive class ratio)

3. Trained `XGBClassifier(n_estimators=300, learning_rate=0.05, max_depth=5, scale_pos_weight=2.77, random_state=42)`
4. Used `eval_set` with early stopping monitoring — printed loss every 50 rounds
5. Generated predictions and probability scores
6. Calculated AUC-ROC — best of all three models
7. Created `compare_all_models()` function loading all 3 saved models
8. Plotted 3 ROC curves on one figure with AUC in legend
9. Saved `reports/figures/12_model_comparison_roc.png`
10. Selected XGBoost as production model — highest AUC
11. Saved `models/xgboost_churn.pkl` and `models/production_model.pkl`
12. Committed all files to GitHub

■ Concepts Learned

- Gradient Boosting: trees trained sequentially — each corrects errors of the previous
- Boosting vs Bagging: boosting is sequential (XGBoost), bagging is parallel (RF)
- `scale_pos_weight`: XGBoost's mechanism for handling imbalanced classes
- `learning_rate` (eta): smaller = more careful steps = better generalisation (usually)
- `subsample` and `colsample_bytree`: randomness in XGBoost to prevent overfitting
- Why XGBoost wins on tabular data: built-in regularisation + gradient optimisation
- Production model alias: saving as `production_model.pkl` decouples code from algorithm name

■ Tools / Software Used

Tool / Library	Version / Notes	Status
XGBoost	2.0.3	✓ Trained with 300 estimators

■ Outputs / Deliverables Produced

- ◆ `src/models/xgboost_model.py` — XGBoost training + model comparison
- ◆ `models/xgboost_churn.pkl` — XGBoost model
- ◆ `models/production_model.pkl` — alias for best model (XGBoost)
- ◆ `reports/figures/12_model_comparison_roc.png` — 3-model ROC curves
- ◆ Model selection documented: XGBoost wins (AUC ~0.86)
- ◆ GitHub commit with XGBoost model and comparison chart

■ Problems Encountered & Solutions

Problem Faced	How I Solved It
XGBoost warning: <code>use_label_encoder</code> deprecated	Added <code>use_label_encoder=False</code> and <code>eval_metric='logloss'</code> explicitly to suppress warning

■ GitHub Commit Record

Commit Message	Week2-Day5: XGBoost trained, 3-model comparison complete, production model selected and saved
Branch	main
Files Changed	src/models/xgboost_model.py, models/xgboost_churn.pkl, models/production_model.pkl, reports/figures/12_model_comparison_roc.png
Commit Hash	(fill after push)

■ End-of-Day Submission Checklist

■ XGBoost trained (300 trees)	■ scale_pos_weight = 2.77 set
■ AUC > RF and LR	■ 3-model ROC chart saved
■ XGBoost selected for production	■ production_model.pkl saved
■ xgboost_churn.pkl saved	■ Chart 12 saved
■ No use_label_encoder warning	■ Day 12 commit on GitHub

■ Self Assessment

Week 2, Day 5			
9/10	6		12/28
Self Rating	Hours Spent	Progress	Day in Program

■ Key Learnings & Reflections

- ★ XGBoost consistently outperforms on structured tabular data — wins again here
- ★ scale_pos_weight is the XGBoost equivalent of class_weight='balanced'
- ★ Saving as 'production_model.pkl' means we can swap the algorithm later without changing the API code
- ★ AUC of ~0.86 means: 86% chance the model ranks a real churner above a non-churner

■ Plan for Tomorrow

- Implement 5-fold cross-validation for more reliable AUC estimate
- Optimise classification threshold using F1 score
- Plot Precision-Recall curve
- Save optimal_threshold.pkl

INTERN SIGNATURE	MENTOR SIGNATURE	DATE SUBMITTED